

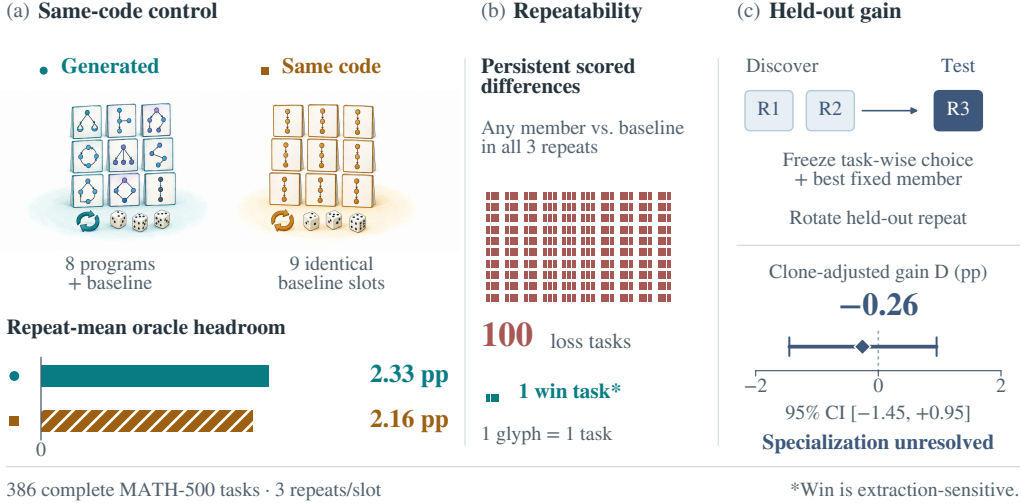


Figure 1: Controlled diagnosis of program specialization. Both arms use 386 complete MATH tasks, nine slots, and three repeats. Headroom uses repeat means. Persistent tasks have at least one generated member winning/losing against baseline in all three repeats; the sole win is extraction-sensitive. D is clone-adjusted held-out repeat gain (95% bootstrap CI). Program drawings are schematic.

The evidence yields three contributions. **First**, identical programs produce positive repeat-averaged oracle headroom, so that signal alone cannot identify program specialization. **Second**, generated programs have substantially more repeatable scored-outcome patterns than clones, dominated by persistent losses: losses span 100 tasks (570 member–task pairs), whereas persistent wins occur on one task. Stable differences therefore need not provide useful task advantages; the clone-adjusted held-out gain is -0.26 pp. **Third**, the frozen selector gains 0.00 pp, and the 27-execution oracle replay ties at 98.70%. On this panel, more programs produce neither a gain for the tested selector nor greater full-budget oracle coverage than more executions of the baseline.

2 RELATED WORK

Automated agent and harness design. ADAS treats agent programs as a search space (Hu et al., 2024). Self-Harness mines failures from traces, proposes targeted edits, and retains edits after regression testing (Zhang et al., 2026b); TTHE adapts the executable harness during evaluation using unlabeled traces (Nie et al., 2026). Other systems improve task adaptation and validation allocation (Zhang et al., 2026a; Miyai et al., 2026), or make harness behavior observable and editable through verification, diagnosis, localization, and policy evolution (Xu et al., 2026; Chen et al., 2026a; Lin et al., 2026; Luo et al., 2026; Wang et al., 2026; Chen et al., 2026b). El et al. (2025) find low diversity in task-level score patterns among the automatically designed agents they study. We separate two explanations for population-level results: distinct programs can execute the same behavior, and identical programs can produce different outcomes. Trace inspection tests whether proposed mechanisms execute; repeated same-code controls isolate the coverage available without changing the program. This connects program equivalence and mutation testing (Budd & Angluin, 1982; Papadakis et al., 2019) to performance attribution in generated harness populations.